

The Fastest IPC You're Afraid to Use

Anurag Jain | Sr. Computer Scientist | Adobe
Illustrator



Shared Memory is Fast

Shared Memory is Fast
So why doesn't everyone
use it?

Pointers don't cross

Pointers don't cross

Different Lifetime semantics

Kernel | File | Process

Fragile locks

Shared objects can outlive its process

Crash

Resource exhaustion

Name collision

Unsafe Reuse

Crash

Resource exhaustion

Name collision

Unsafe Reuse

boost::interprocess...

Hard problems are ours to handle...

Raiden

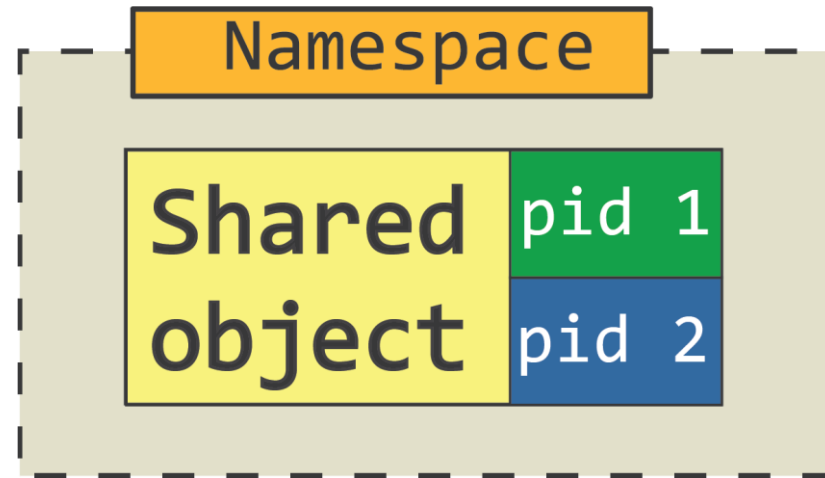
Makes shared
memory safe
to use

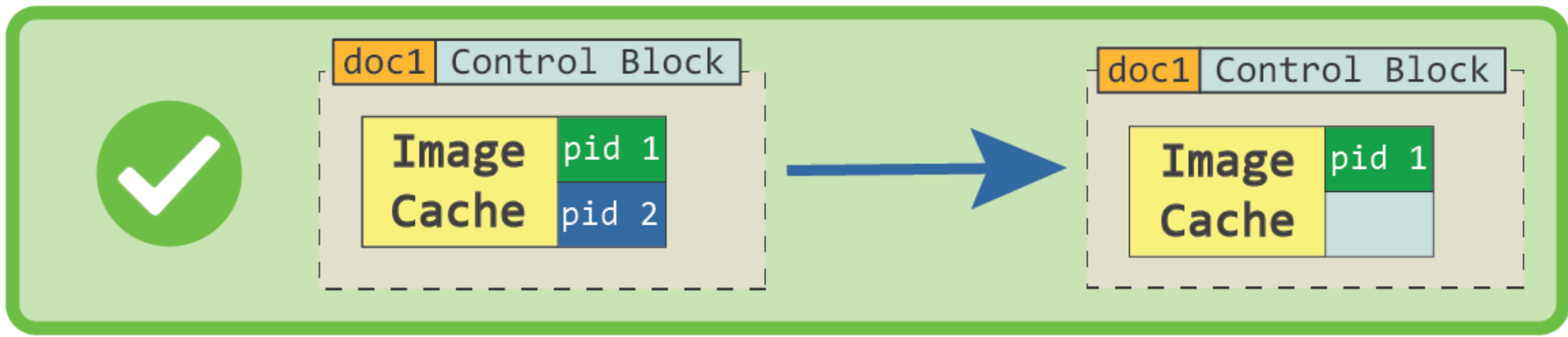


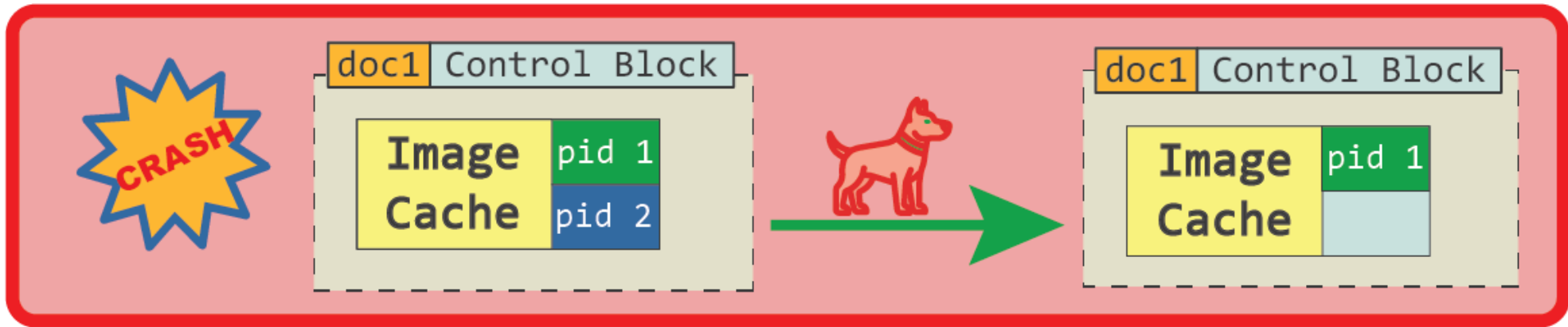
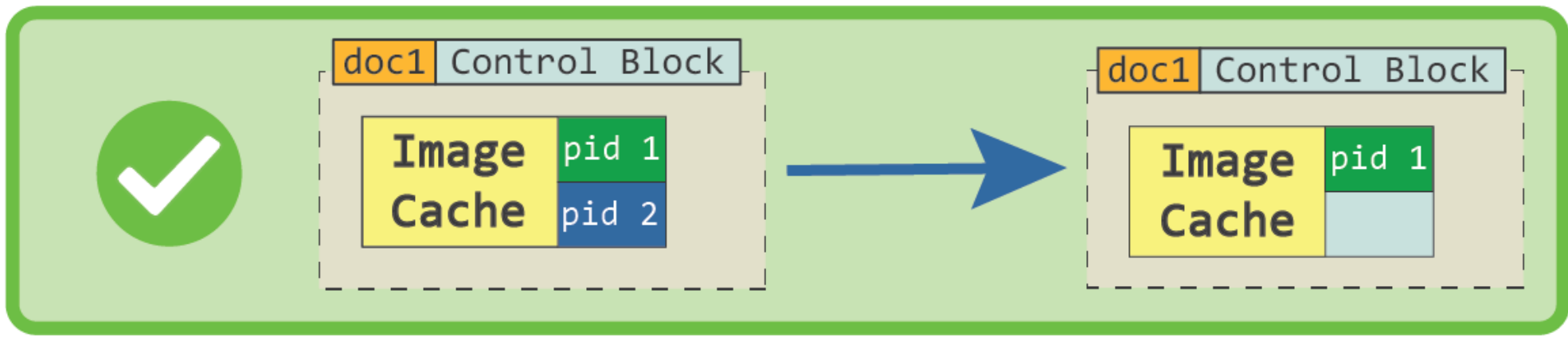
Shipping in
Adobe
Illustrator

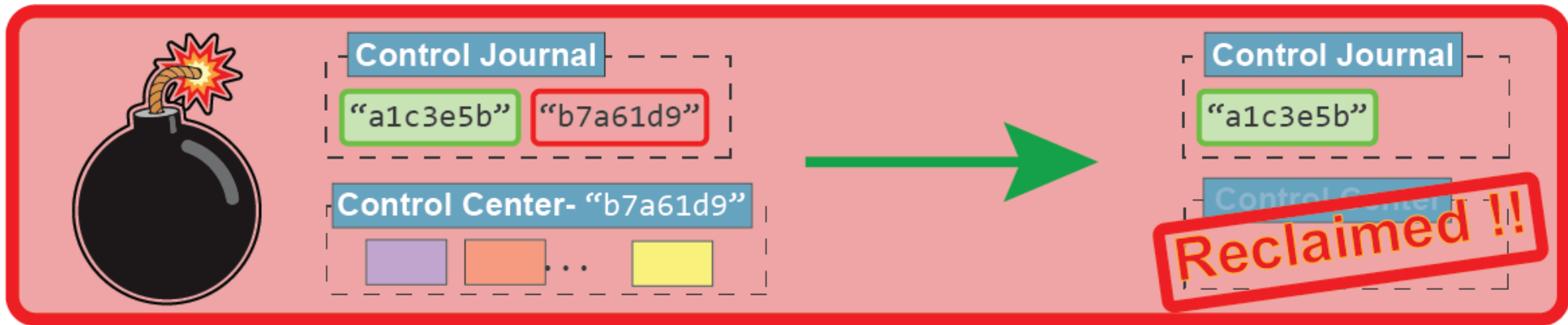
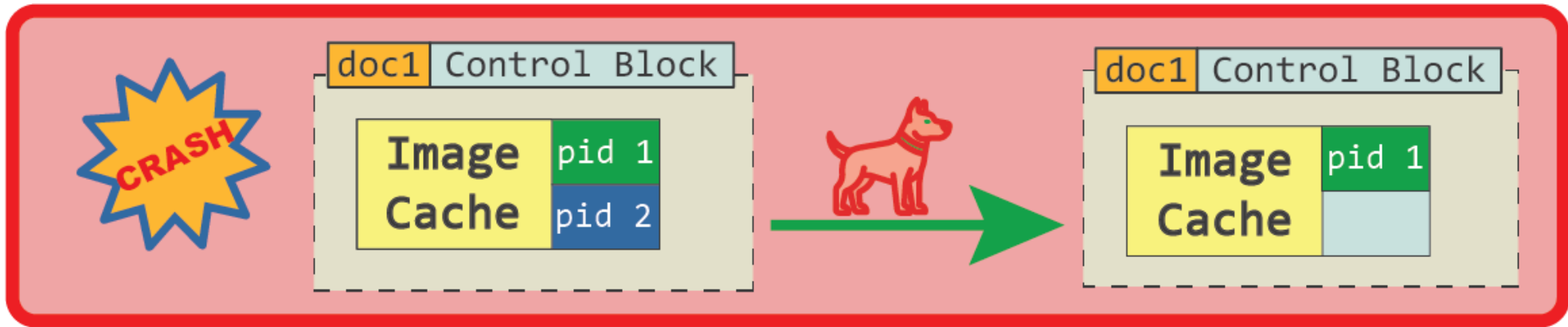
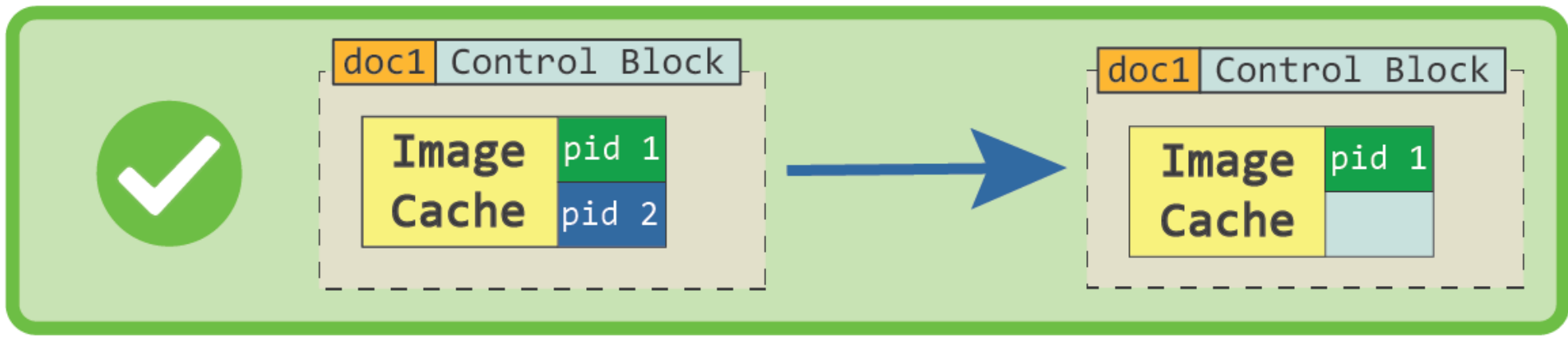
Sessions
Entity
Namespaces

Crash Recovery Cross-process RAI









Poster #2

Life-cycle, reusable design And much more ...



Raiden: Fail-Safe Interprocess Architecture for Large-Scale Desktop Applications

Crash-safe shared-memory lifecycle for multi-process C++ – shipping in Adobe Illustrator

Tarun Beri • Anurag Jain • Kausik Naguri • Shyam Poovalah • Prateek Soni
Adobe Inc.



September 17-18
Aurora, Colorado, USA

Abstract

Raiden, a production-deployed interprocess-communication (IPC) framework that provides robust shared-state management for large, long-lived, multi-process desktop applications. Raiden is built on top of **boost::interprocess** but layers onto it the structure that raw IPC primitives deliberately omit: it organizes all shared resources around a session, exposes hierarchical namespaces that mirror application structure, and gives every shared object an explicit, reference-counted lifecycle with automatic reclamation after abnormal termination. Concretely, Raiden provides: session-scoped shared memory segments, typed, namespace-held entities (shared memory, message queues, mutexes/condition variables, interprocess caches, growable STL-like containers, and growable byte buffers); a cross-process procedure-call mechanism; and fail-safe cleanup of resources left behind by crashed processes.

The Problem

Split a desktop app into cooperating processes and C++ stops helping you at the boundary. Raw OS IPC primitives are powerful but intentionally minimal, leaving the hard problems to you:

- Pointers don't cross.** There is no shared heap; shared_ptr and raw pointers are meaningless in another process.
- Stale, orphaned resources.** Segments and named mutexes are kernel/filesystem-persistent and removed only by explicit calls, so a process that dies leaks them; they pile up into name collisions, resource exhaustion, and unsafe reuse.
- Global names collide.** Flat, globally visible names are shared across runs and even users, so unrelated sessions interfere by accident.
- Locks are fragile under failure.** A holder that dies inside a critical section leaves a named mutex locked forever, hanging every peer.
- Ownership is implicit.** No process clearly owns a resource's lifecycle; scattered create/destroy logic makes safe cleanup guesswork.
- Shared segments can't grow.** A segment's size is fixed once mapped, so growing a shared container means hand-rolled pre-allocation, remapping, or serialization.

Why Raiden?

Raiden is not a new IPC transport. It uses the same OS primitives as Boost and adds the architecture around them:

- Crash-safe cleanup.** Raiden reference-counts every entity and auto-reclaims it using the journal and "PID" = start-time liveness.
- Collision-free naming.** Raiden bakes the session identifier into every OS name, and groups entities in nested, session-scoped namespaces (e.g. /documents/doc1) that mirror application structure.
- Explicit ownership.** Raiden makes exclusive/shared ownership explicit, and with reference counting, the last live owner frees the object.
- Discovery.** Raiden's entities are self-describing and held in a templated registry, so a joining process finds them by name or kind and maps only what it needs, with no out-of-band name passing.
- Managed locks.** Raiden's SysV/NTLW confines the locks to tiny callback-free critical sections, and an orphaned lock is removed at startup to recover OS resources.
- RAII.** Allocations return RAII scope guards; offset_ptr keeps references valid in every process's address space.
- Growable shared data.** ContainerAggregator spans one STL container over many segments, and GrowableInterProcessBuffer is an append-only stream, growing without remapping.

Also: Entity discovery by name or kind (no out-of-band names), type-safe cross-process RPC (variadic templates + index_sequence), and a policy-based cache (LRU / MRU / LFU / MFU).

System Architecture: Shared Memory & Interprocess Discovery

Fail-Safe Cleanup & Stale-Session Reclamation

Internal Components

Completion Status

Completed and in production since Jan 2025. Raiden is fully implemented and shipping inside Adobe Illustrator, a commercial desktop application exceeding 50 million lines of code, and has been exercised across more than 2.4 million production sessions. Illustrator used Raiden to make its transition to a multi-process architecture both robust and fast and presently it underpins several multi-process features:

- Shared imaging and asset caches** built on the interprocess cache and shared-memory entities, enabling zero-copy re-use of decoded images and assets across co-operating processes.
- Auto file save, export, and document crash recovery:** document modification state is periodically shared over a growable interprocess buffer, including on-disk serialization/restore of large document state.
- Render/tile pipelines** built on container aggregators and shared memory for dynamically growing shared datasets.

Raiden runs on macOS and Windows in production, with experimental support for additional platforms.

Benchmarks

The tables below summarize results from controlled benchmarks, where Table 1 and 2 report mean per-operation latency (1000 iterations; 100 for cross-process open) in microseconds; Table 3 and 4 report container aggregator benchmarks with CPU time over 100K operations in milliseconds.

Table 1: Framework overhead – Raiden only (µs)

Operation	Boost	Raiden	Diff
Init: Create Journal in Center = ROOT	1932.30	1932.30	0.0%
Namespace creation (one time)	1.30	1.30	0.0%
Namespace creation (10K)	1.61	1.61	0.0%
Namespace creation (chunked)	1.61	1.61	0.0%
Namespace creation (single)	20.45	20.45	0.0%
Message queue creation	1.47	1.47	0.0%
Message queue (max 10)	52.00	52.00	0.0%
Message queue lookup	12.71	12.71	0.0%

Table 2: Sequential aggregators vs. Boost (100K ops ms)

Operation	Boost	Raiden	Diff
Vector	0.215	0.215	+0.5%
push_back	0.056	0.056	+73.2%
Random access	0.056	0.056	+73.2%
Clear (one large)	554.22	554.90	+0.1%

Table 3: Associative aggregators vs. Boost (100K ops ms)

Operation	Boost	Raiden	Diff
Set	20.907	21.588	+3.3%
Find	14.388	14.485	+0.7%
Map	21.116	21.874	+3.6%
Insert	14.654	14.707	+0.4%
Delete	23.060	23.724	+2.9%

Adobe Inc. | Illustrator Engineering
beri • anuraj • knaguri • spoovalah • psoni @adobe.com

References & Acknowledgments
<https://www.boost.org/doc/libs/latest/doc/html/interprocess.html>

13